Name: Prabhkirat Singh Marwah

SECTION: B

BATCH: B4

Roll no: 47

ML LAB 9

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans,AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import (silhouette_score)

from scipy.cluster.hierarchy import linkage,dendrogram

sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize']=(8,6)

df=pd.read_csv("WHOLESALE.csv")
```

In [2]:
```
df.head()
```

Out[2]:

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	2	3	12669	9656	7561	214	2674	1338
1	2	3	7057	9810	9568	1762	3293	1776
2	2	3	6353	8808	7684	2405	3516	7844
3	1	3	13265	1196	4221	6404	507	1788
4	2	3	22615	5410	7198	3915	1777	5185

In [3]:
```
df.tail()
```

Out[3]:

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
435	1	3	29703	12051	16027	13135	182	2204
436	1	3	39228	1431	764	4510	93	2346
437	2	3	14531	15488	30243	437	14841	1867
438	1	3	10290	1981	2232	1038	168	2125
439	1	3	2787	1698	2510	65	477	52

In [4]:
```
df.shape
```

Out[4]:
```
(440, 8)
```

In [5]:
```
df.columns
```

Out[5]:
```
Index(['Channel', 'Region', 'Fresh', 'Milk', 'Grocery', 'Frozen',
       'Detergents_Paper', 'Delicassen'],
      dtype='object')
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Channel           440 non-null    int64
 1   Region            440 non-null    int64
 2   Fresh             440 non-null    int64
 3   Milk              440 non-null    int64
 4   Grocery           440 non-null    int64
 5   Frozen            440 non-null    int64
 6   Detergents_Paper  440 non-null    int64
 7   Delicassen        440 non-null    int64
dtypes: int64(8)
memory usage: 27.6 KB
```

In [7]:
```
df.describe()
```

Out[7]:

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
count	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000	440.000000
mean	1.322727	2.543182	12000.297727	5796.265909	7951.277273	3071.931818	2881.493182	1524.870455
std	0.468052	0.774272	12647.328865	7380.377175	9503.162829	4854.673333	4767.854448	2820.105937
min	1.000000	1.000000	3.000000	55.000000	3.000000	25.000000	3.000000	3.000000
25%	1.000000	2.000000	3127.750000	1533.000000	2153.000000	742.250000	256.750000	408.250000
50%	1.000000	3.000000	8504.000000	3627.000000	4755.500000	1526.000000	816.500000	965.500000
75%	2.000000	3.000000	16933.750000	7190.250000	10655.750000	3554.250000	3922.000000	1820.250000
max	2.000000	3.000000	112151.000000	73498.000000	92780.000000	60869.000000	40827.000000	47943.000000

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
Channel            0
Region             0
Fresh              0
Milk               0
Grocery            0
Frozen             0
Detergents_Paper   0
Delicassen         0
dtype: int64
```

In [9]:
```
df.duplicated().sum()
```

Out[9]:
```
np.int64(0)
```

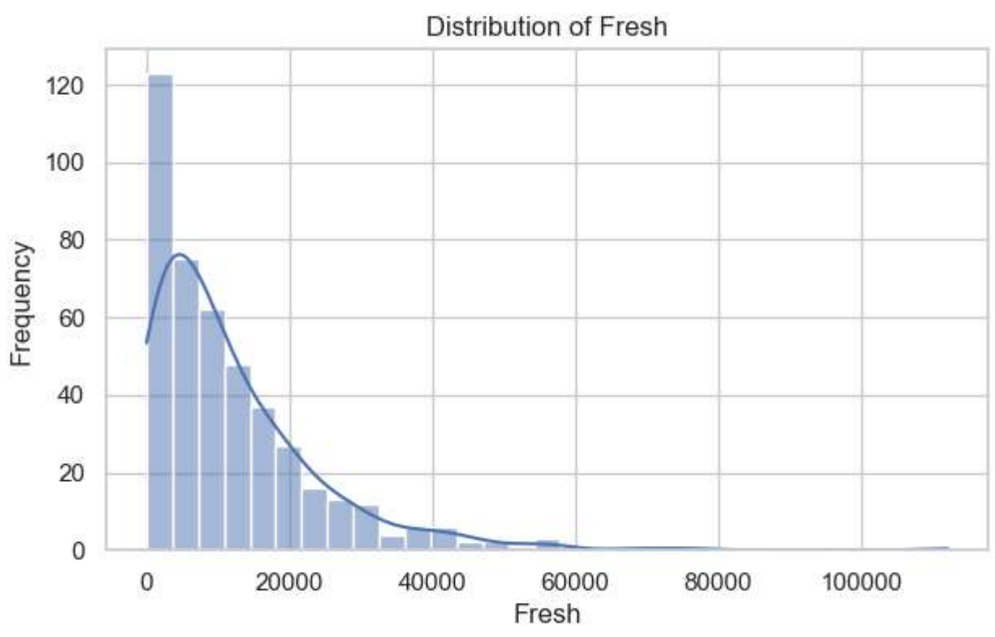
In [10]:
```
df.dtypes
```

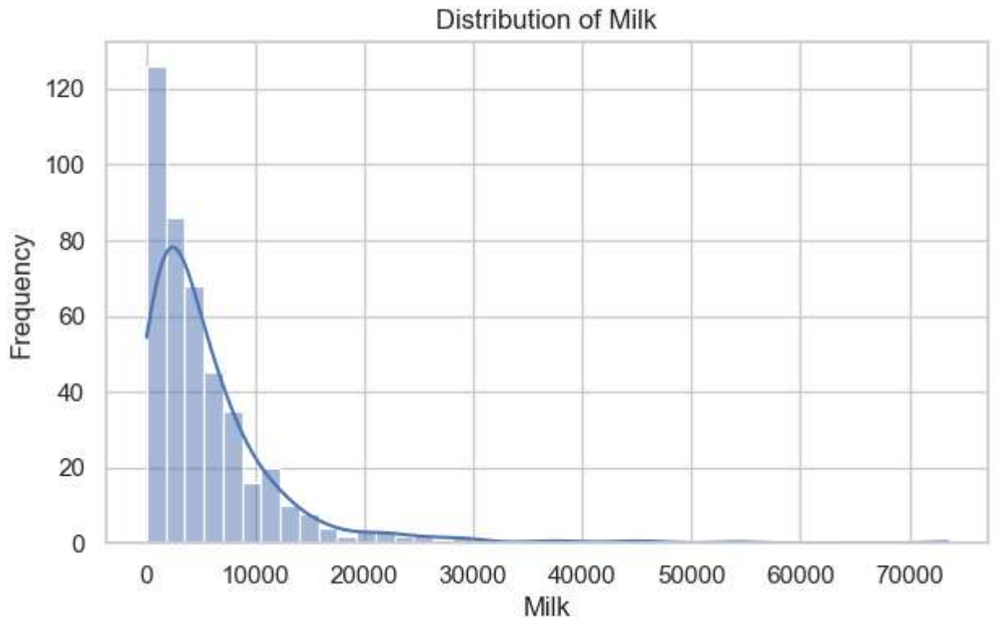
Out[10]:
```
Channel            int64
Region             int64
Fresh              int64
Milk               int64
Grocery            int64
Frozen             int64
Detergents_Paper   int64
Delicassen         int64
dtype: object
```

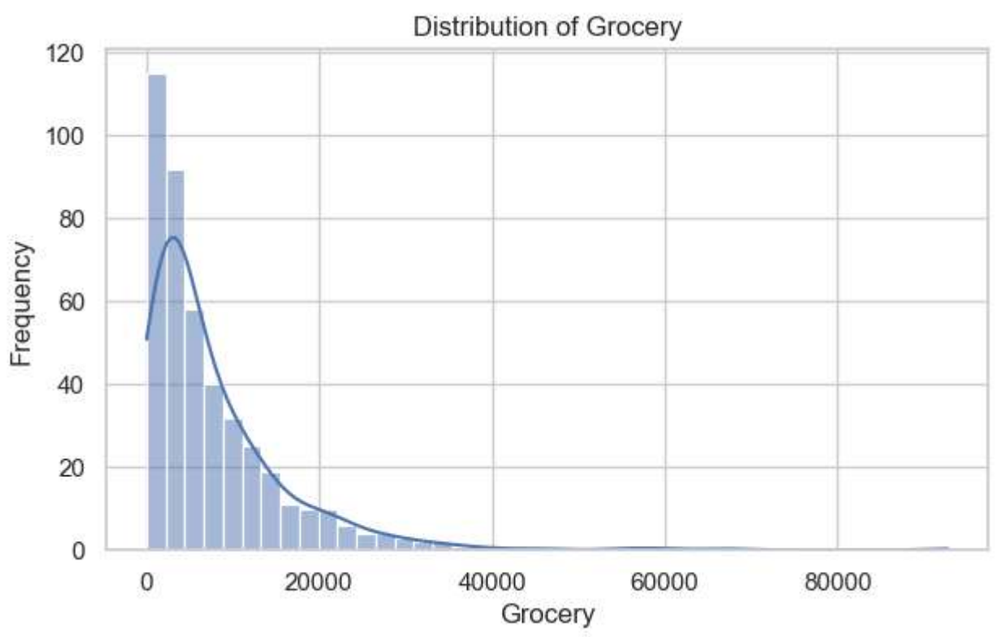
Univariate Analysis

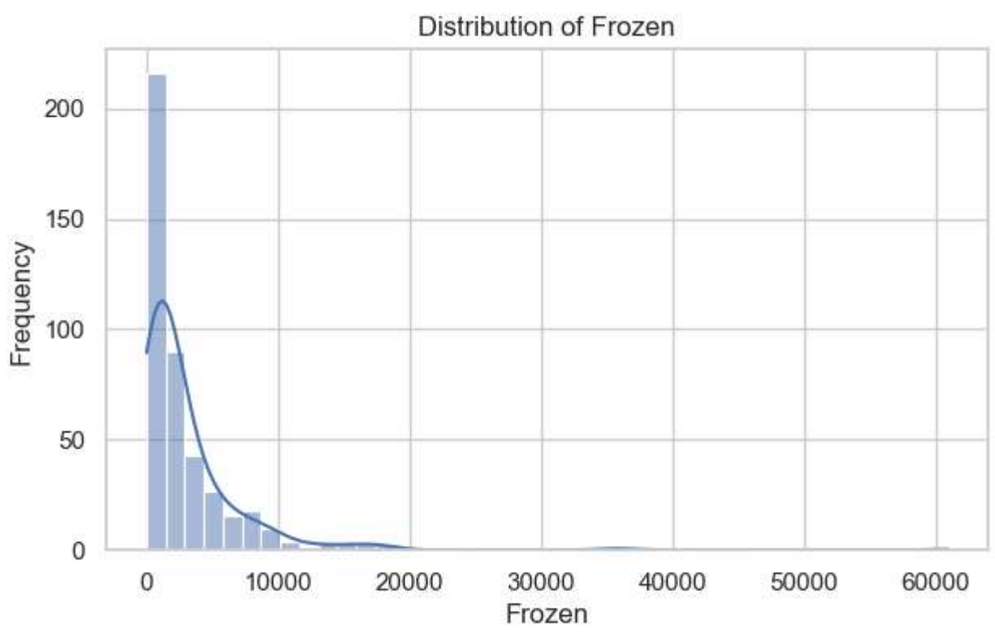
In [11]:
```
#Histogram+KDE
numeric_cols=df.drop(columns=['Channel','Region']).columns

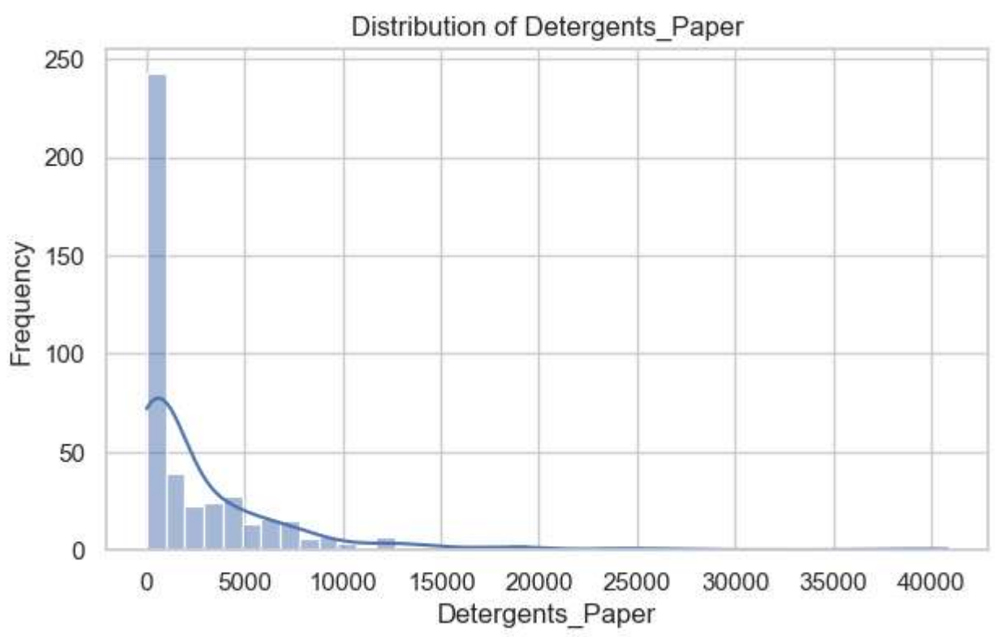
for col in numeric_cols:
    plt.figure(figsize=(7,4))
    sns.histplot(df[col],kde=True)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()
```

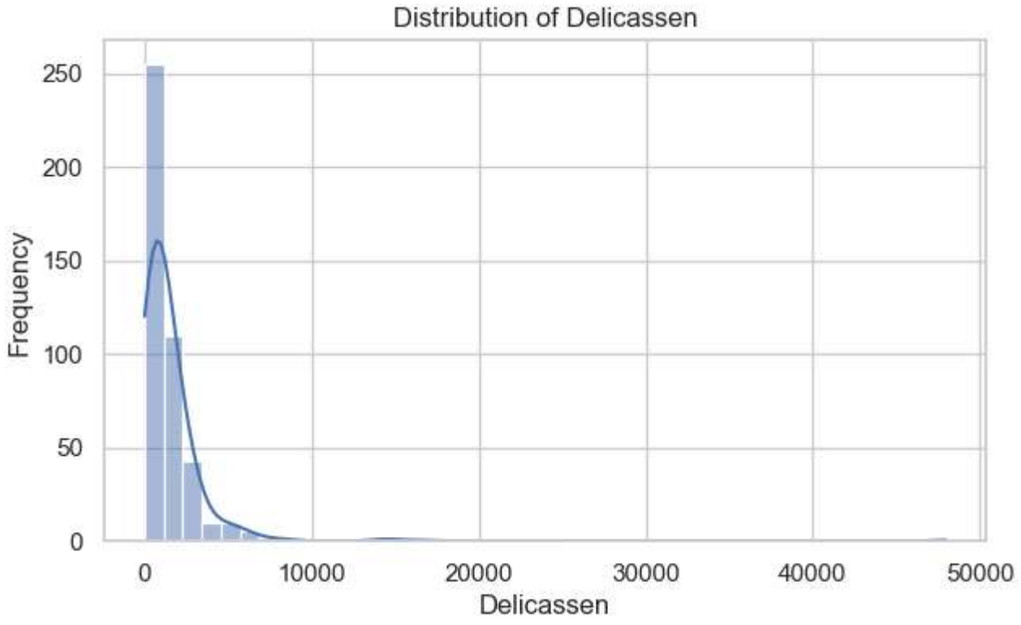

Distribution of Fresh
Frequency
120
100
80
60
40
20
0
0
20000
40000
60000
80000
100000
Fresh


Distribution of Milk
Frequency
120
100
80
60
40
20
0
0
10000
20000
30000
40000
50000
60000
70000
Milk

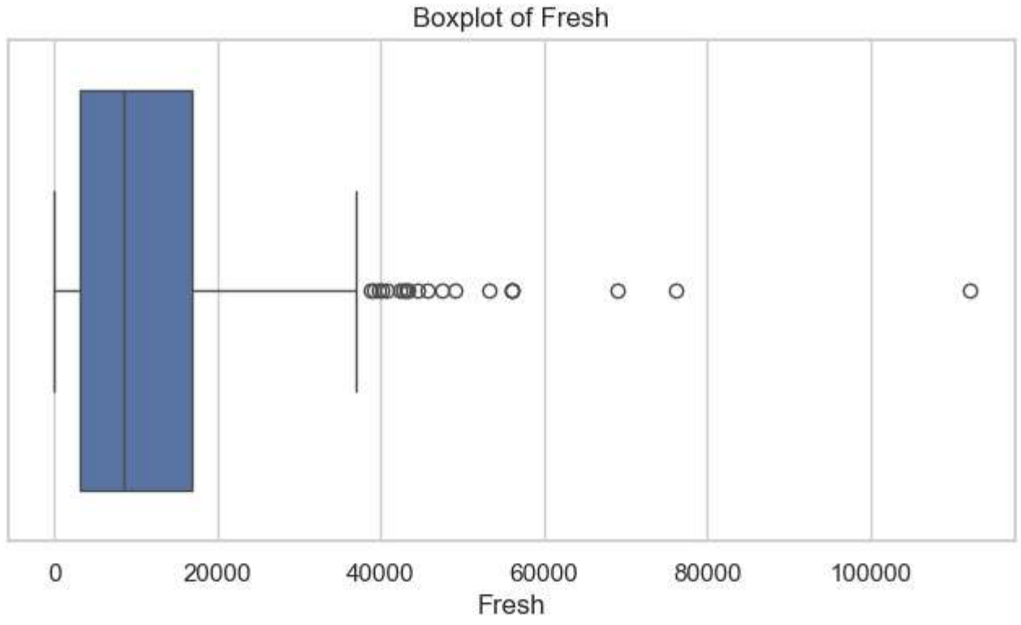

Distribution of Grocery
Frequency
120
100
80
60
40
20
0
0
20000
40000
60000
80000
Grocery

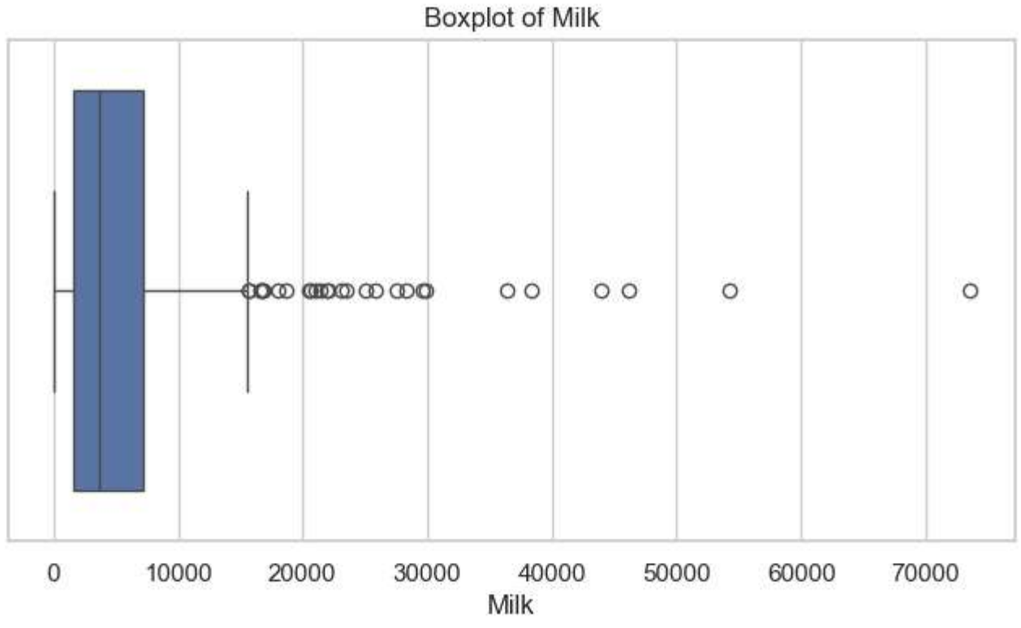

Distribution of Frozen
Frequency
200
150
100
50
0
0
10000
20000
30000
40000
50000
60000
Frozen

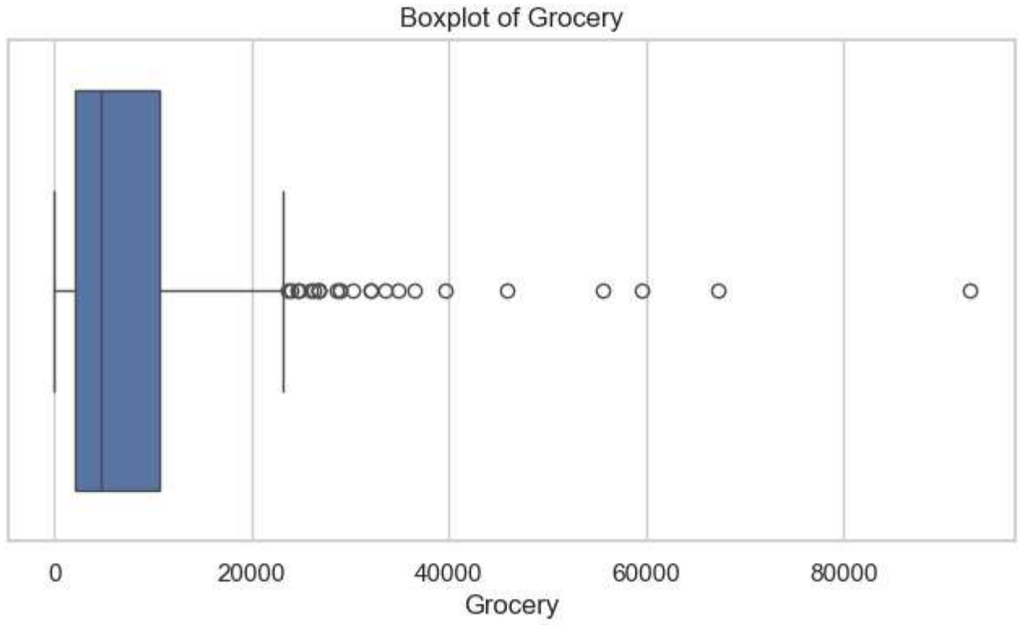

Distribution of Detergents_Paper
Frequency
250
200
150
100
50
0
0
5000
10000
15000
20000
25000
30000
35000
40000
Detergents_Paper

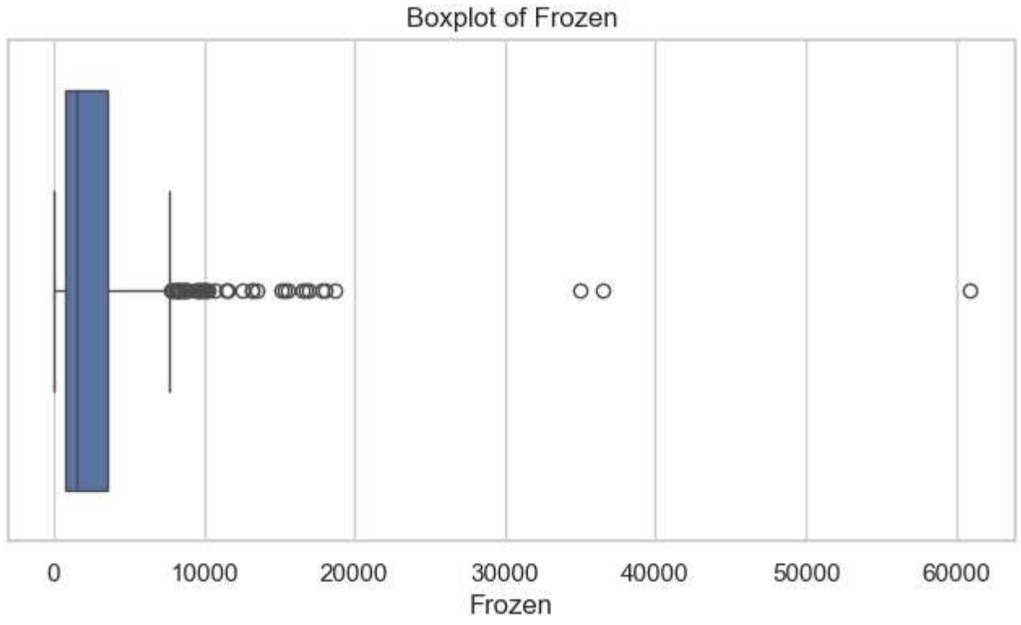


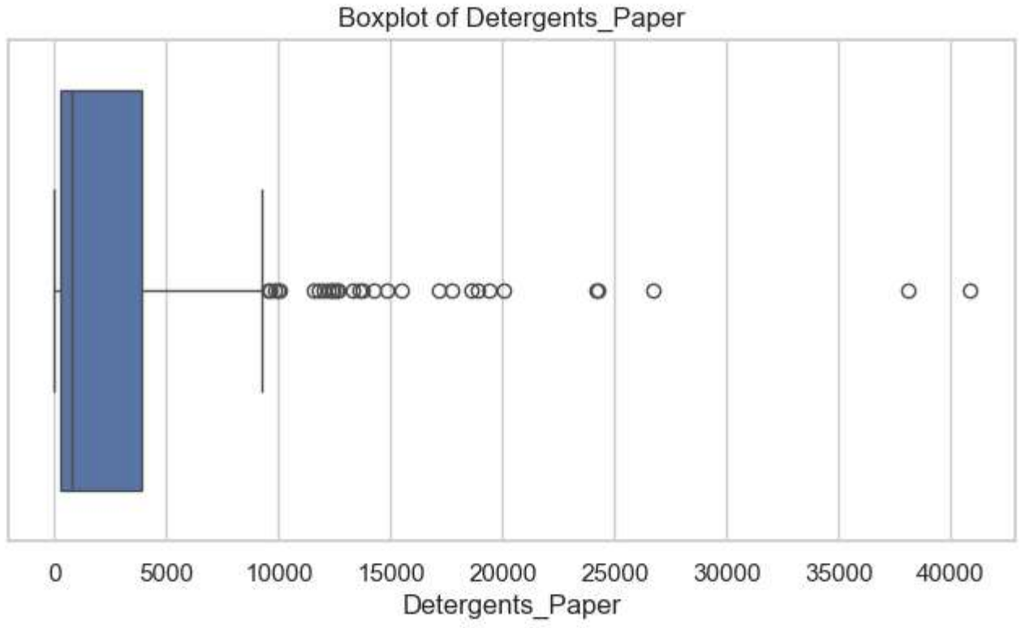
In [12]:
```
# Boxplot-Outlier Analysis
for col in numeric_cols:
    plt.figure(figsize=(8,4))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()
```

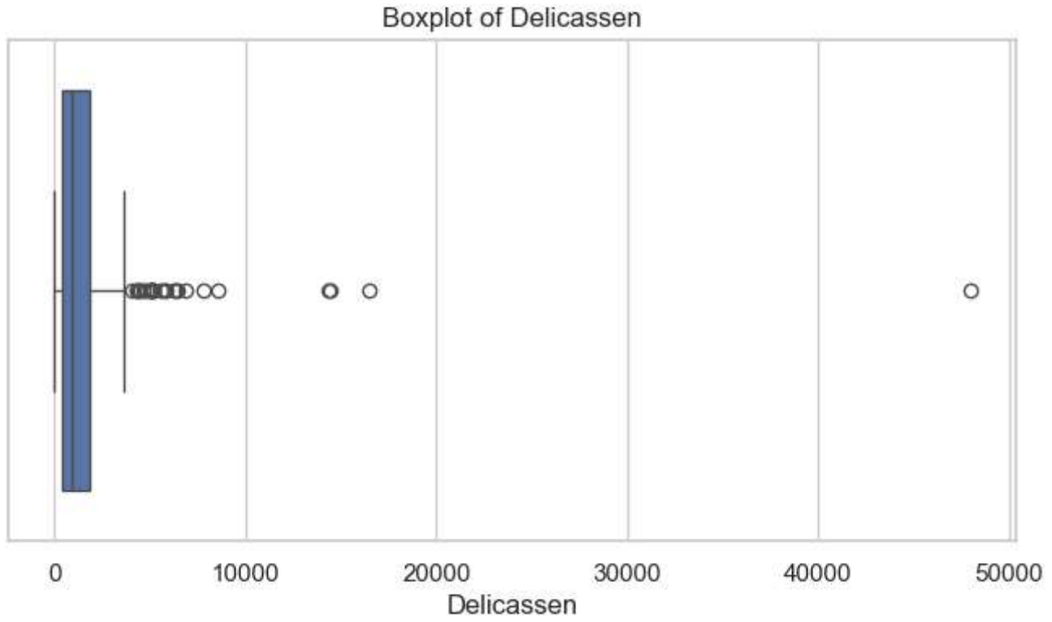

Boxplot of Fresh
0
20000
40000
60000
80000
100000
Fresh


Boxplot of Milk
0
10000
20000
30000
40000
50000
60000
70000
Milk


Boxplot of Grocery
0
20000
40000
60000
80000
Grocery


Boxplot of Frozen
0
10000
20000
30000
40000
50000
60000
Frozen


Boxplot of Detergents_Paper
0
5000
10000
15000
20000
25000
30000
35000
40000
Detergents_Paper



In [13]:
```python
# Detect outliers for every feature
features=df.drop(columns=["Channel","Region"])

#Calculate Q1 and Q3
Q1=features.quantile(0.25)
Q3=features.quantile(0.75)

# Calculate IQR
IQR=Q3-Q1

#Calculate Lower and Upper Limits
lower_bound=Q1-1.5*IQR
upper_bound=Q3+1.5*IQR

# Detect Outliers
outlier_mask=(features<lower_bound) | (features>upper_bound)

#Count outliers for each feature
```

```
outlier_count=outlier_mask.sum()

#Percentage of Outliers
outlier_percentage=(outlier_count/len(features))*100

#Create Summary table
outlier_summary=pd.DataFrame({
    'Q1':Q1,
    'Q3':Q3,
    'IQR':IQR,
    'lower bound':lower_bound,
    'upper bound':upper_bound,
    'Outlier Count':outlier_count,
    'Outlier %':outlier_percentage
})

outlier_summary.round(3)
```

Out[13]:

	Q1	Q3	IQR	lower bound	upper bound	Outlier Count	Outlier %
Fresh	3127.75	16933.75	13806.00	-17581.250	37642.750	20	4.545
Milk	1533.00	7190.25	5657.25	-6952.875	15676.125	28	6.364
Grocery	2153.00	10655.75	8502.75	-10601.125	23409.875	24	5.455
Frozen	742.25	3554.25	2812.00	-3475.750	7772.250	43	9.773
Detergents_Paper	256.75	3922.00	3665.25	-5241.125	9419.875	30	6.818
Delicassen	408.25	1820.25	1412.00	-1709.750	3938.250	27	6.136

In [14]:

```
#No of outliers in each feature

print("Outliers in each feature:\n")

for column in features.columns:
    print(f"{column:25s}:{outlier_count[column]}")
```

```
Outliers in each feature:

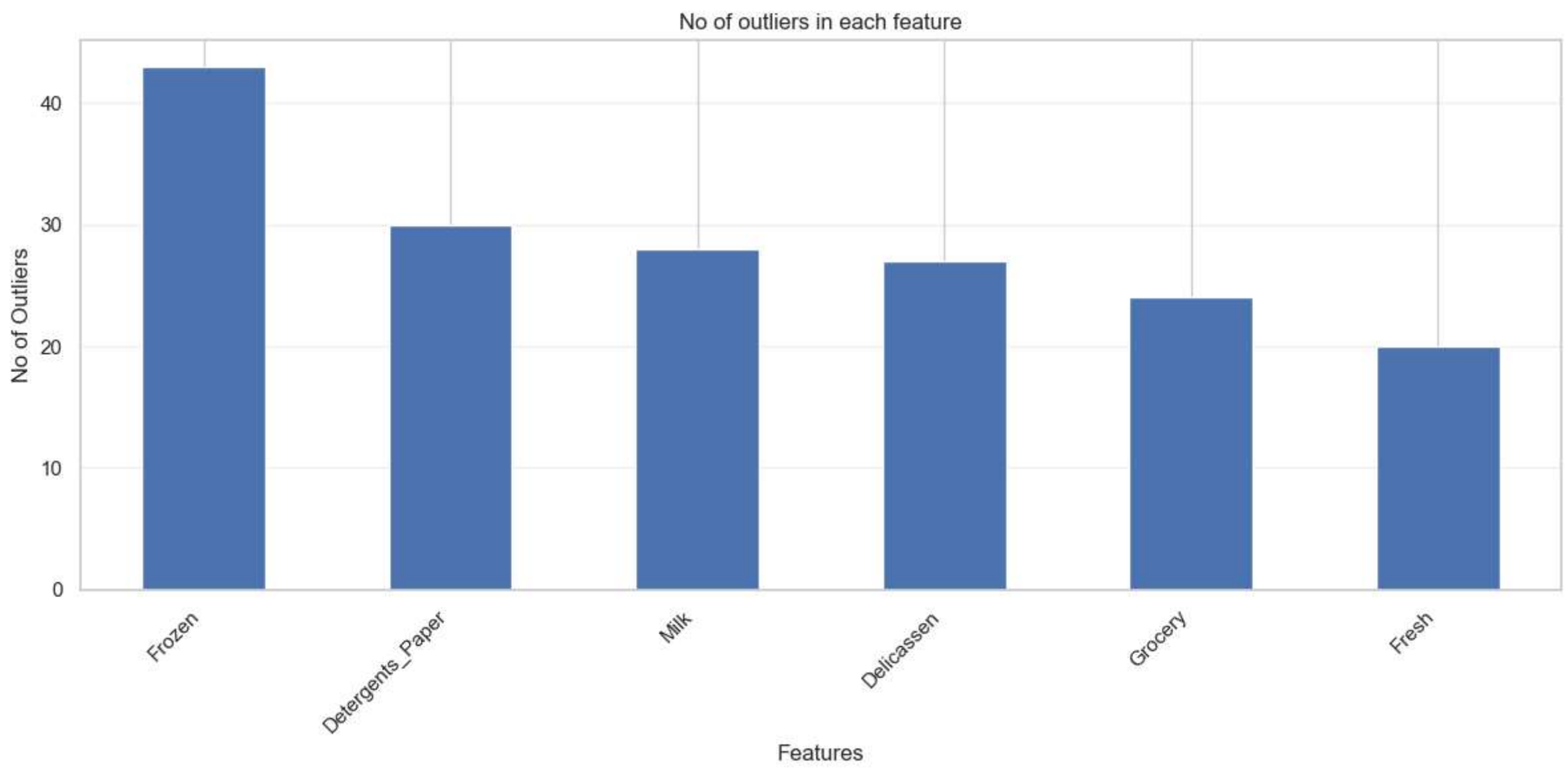
Fresh                   :20
Milk                    :28
Grocery                 :24
Frozen                  :43
Detergents_Paper        :30
Delicassen              :27
```

In [15]:
```
plt.figure(figsize=(12,6))

outlier_count.sort_values(ascending=False).plot(
    kind='bar'
)

plt.title("No of outliers in each feature")
plt.xlabel("Features")
plt.ylabel("No of Outliers")
plt.xticks(rotation=45,ha='right')
plt.grid(axis='y',alpha=0.3)

plt.tight_layout()
plt.show()
```



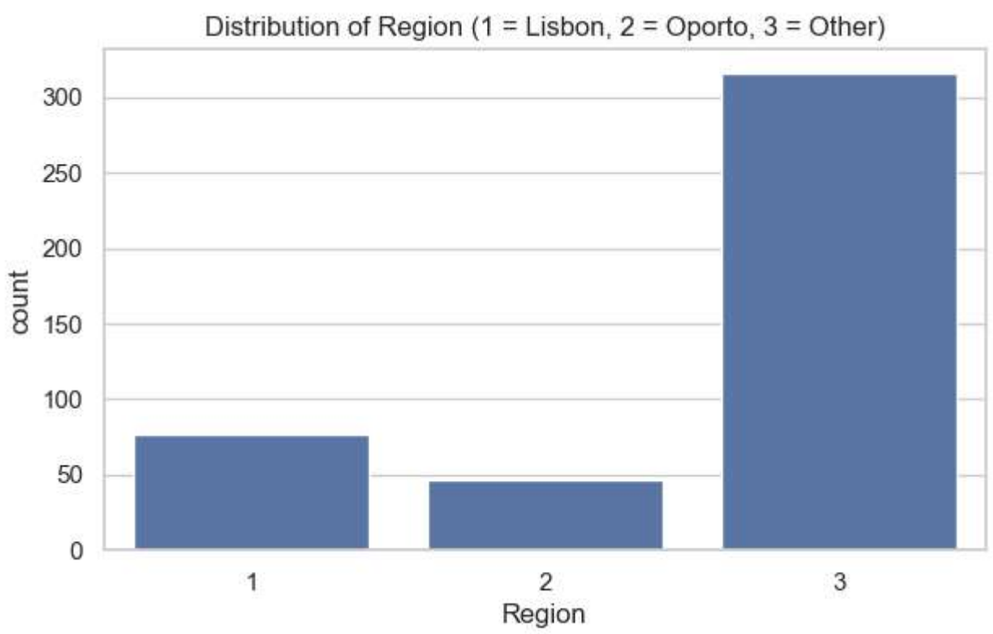
In [16]:
```
#Channel Distribution
plt.figure(figsize=(7,4))
sns.countplot(x='Channel',data=df)
plt.title("Distribution of Channel (1 = Horeca, 2 = Retail)")
plt.show()
```



In [17]:
```
df['Channel'].value_counts().sort_index()
```

Out[17]:
```
Channel
1    298
2    142
Name: count, dtype: int64
```

In [18]:
```
#Region Distribution
plt.figure(figsize=(7,4))
sns.countplot(x='Region',data=df)
plt.title("Distribution of Region (1 = Lisbon, 2 = Oporto, 3 = Other)")
plt.show()
```

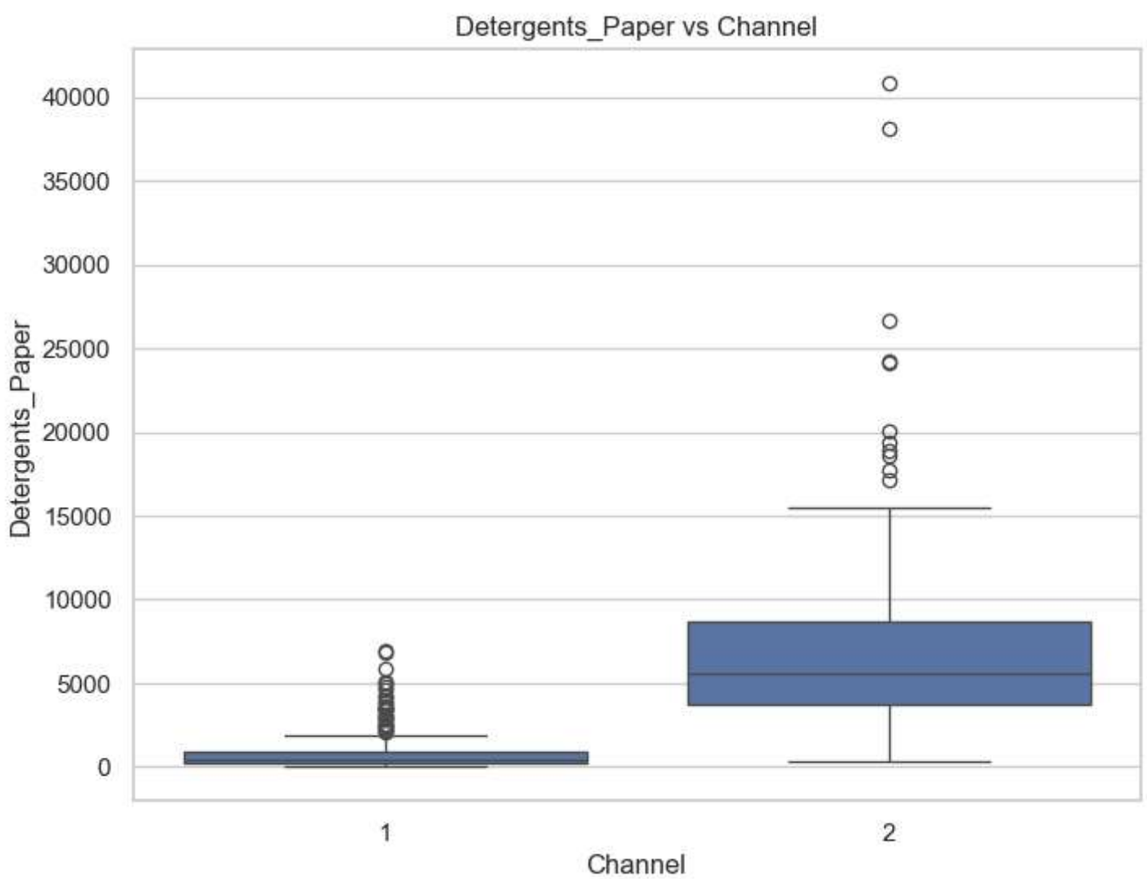


In [19]:
```
df['Region'].value_counts().sort_index()
```

Out[19]:
```
Region
1     77
2     47
3    316
Name: count, dtype: int64
```

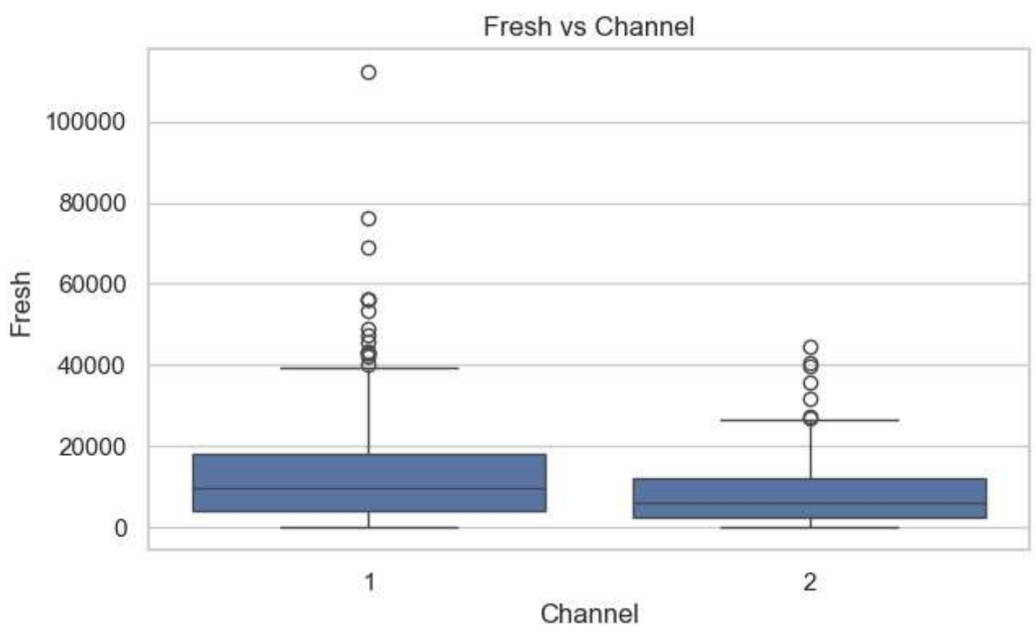
2.Bivariate Analysis

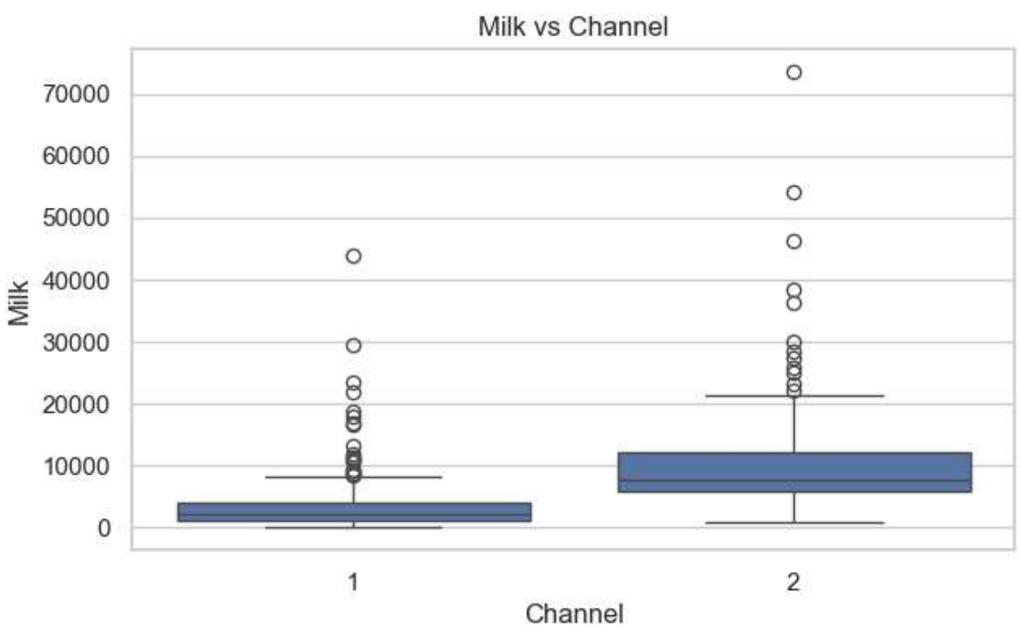
In [20]:
```
#Detergents_Paper vs Channel
sns.boxplot(x='Channel',y='Detergents_Paper',data=df)
plt.title('Detergents_Paper vs Channel')
plt.show()
```

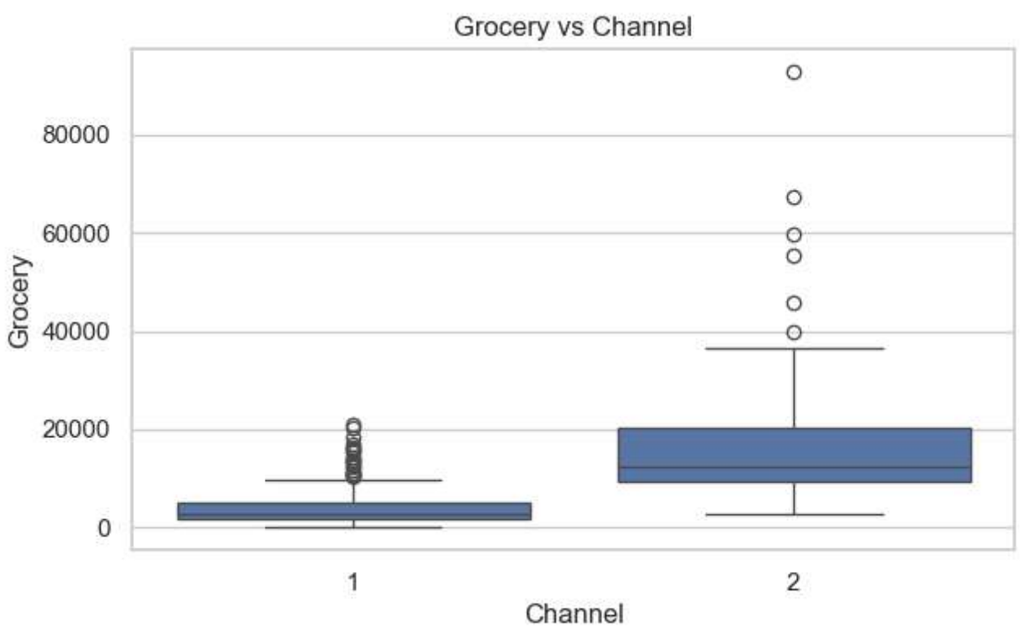


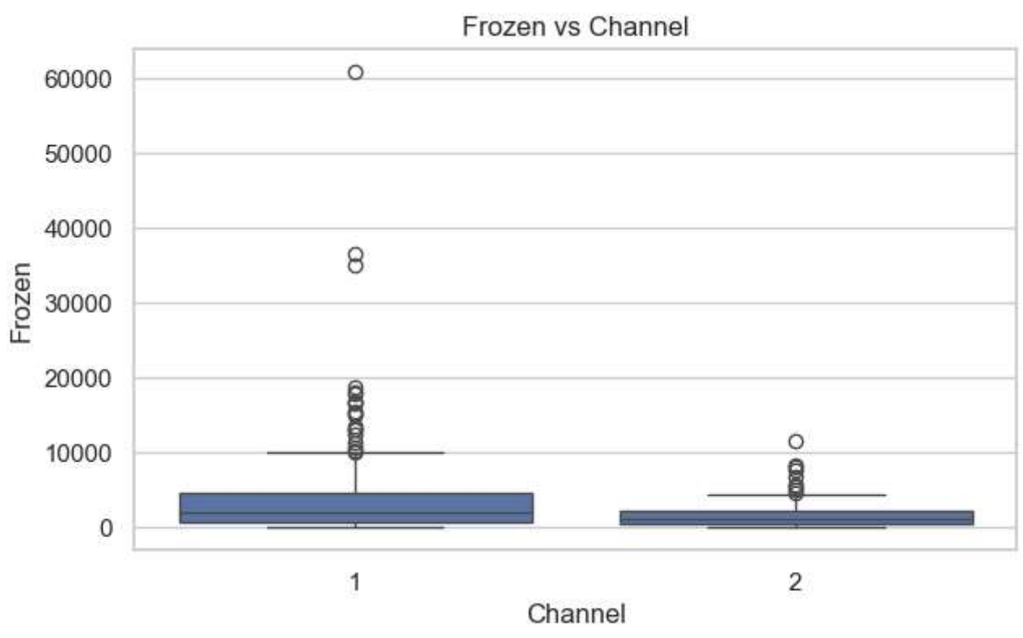
In [21]:
```
spend_features=['Fresh','Milk','Grocery','Frozen','Detergents_Paper','Delicassen']

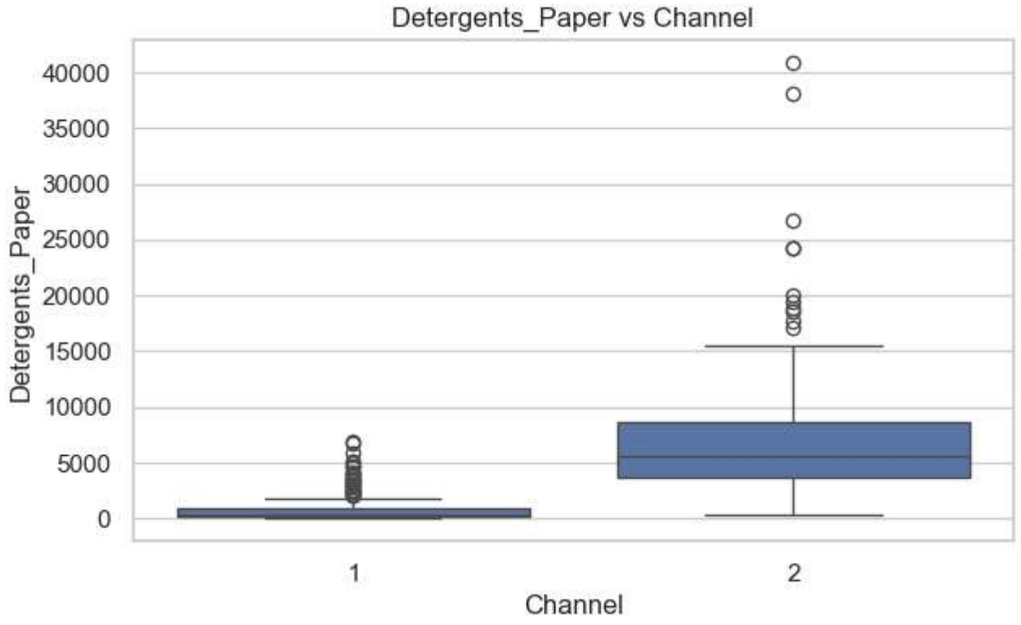
for col in spend_features:
    plt.figure(figsize=(7,4))
    sns.boxplot(x='Channel',y=col,data=df)
    plt.title(f'{col} vs Channel')
    plt.show()
```

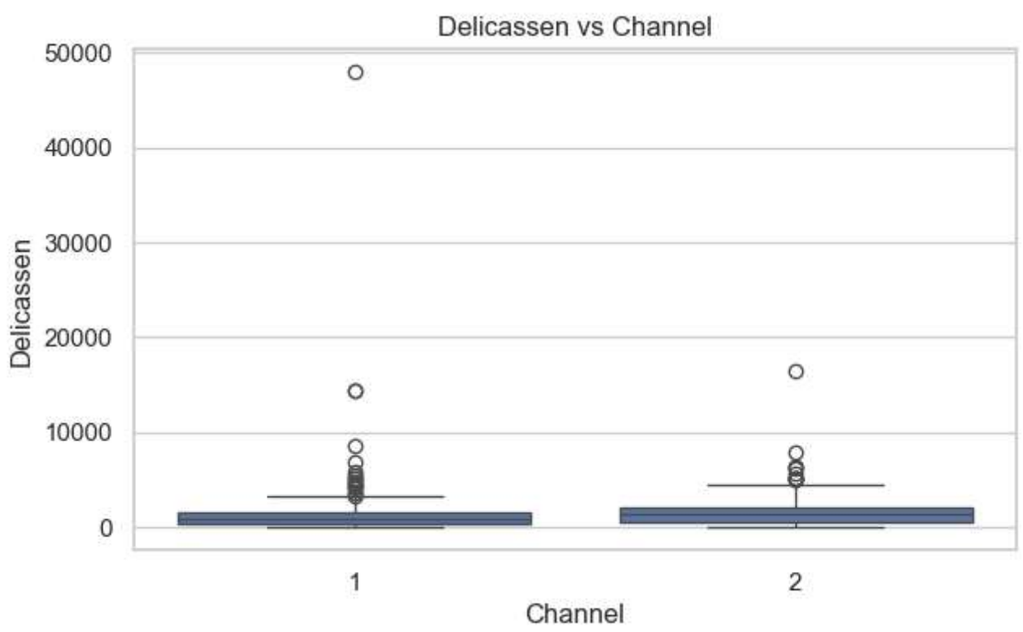

Fresh vs Channel
Fresh
100000
80000
60000
40000
20000
0
1
2
Channel


Milk vs Channel
70000
60000
50000
40000
30000
20000
10000
0
Milk
1
2
Channel


Grocery vs Channel
Grocery
80000
60000
40000
20000
0
1
2
Channel


Frozen vs Channel
60000
50000
40000
30000
20000
10000
0
Frozen
1
2
Channel


Detergents_Paper vs Channel
Detergents_Paper
40000
35000
30000
25000
20000
15000
10000
5000
0
1
2
Channel



In [22]:
```
#Mean spending by Channel and by Region
print("Mean spending by Channel")
display(df.groupby('Channel')[spend_features].mean().round(1))

print("Mean spending by Region")
display(df.groupby('Region')[spend_features].mean().round(1))
```

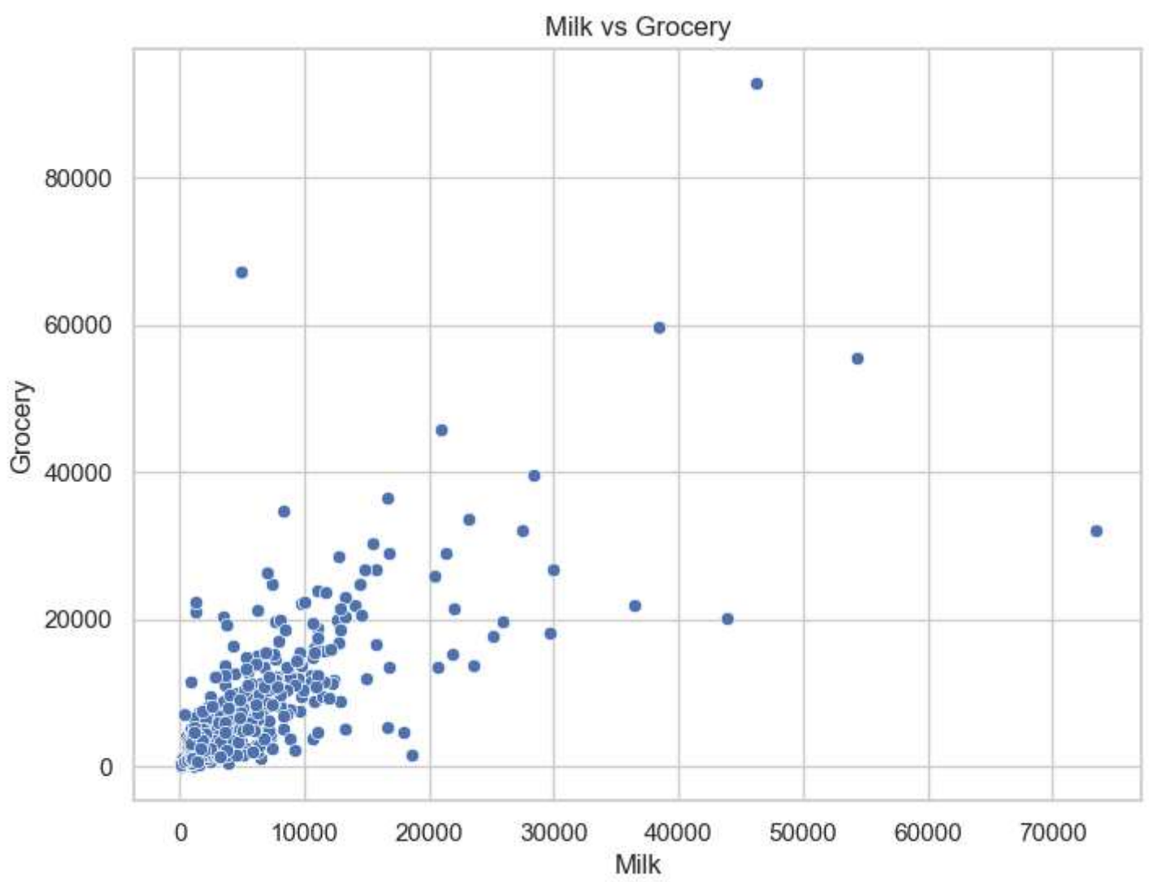
```
Mean spending by Channel
```

Channel	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
1	13475.6	3451.7	3962.1	3748.3	790.6	1416.0
2	8904.3	10716.5	16322.9	1652.6	7269.5	1753.4

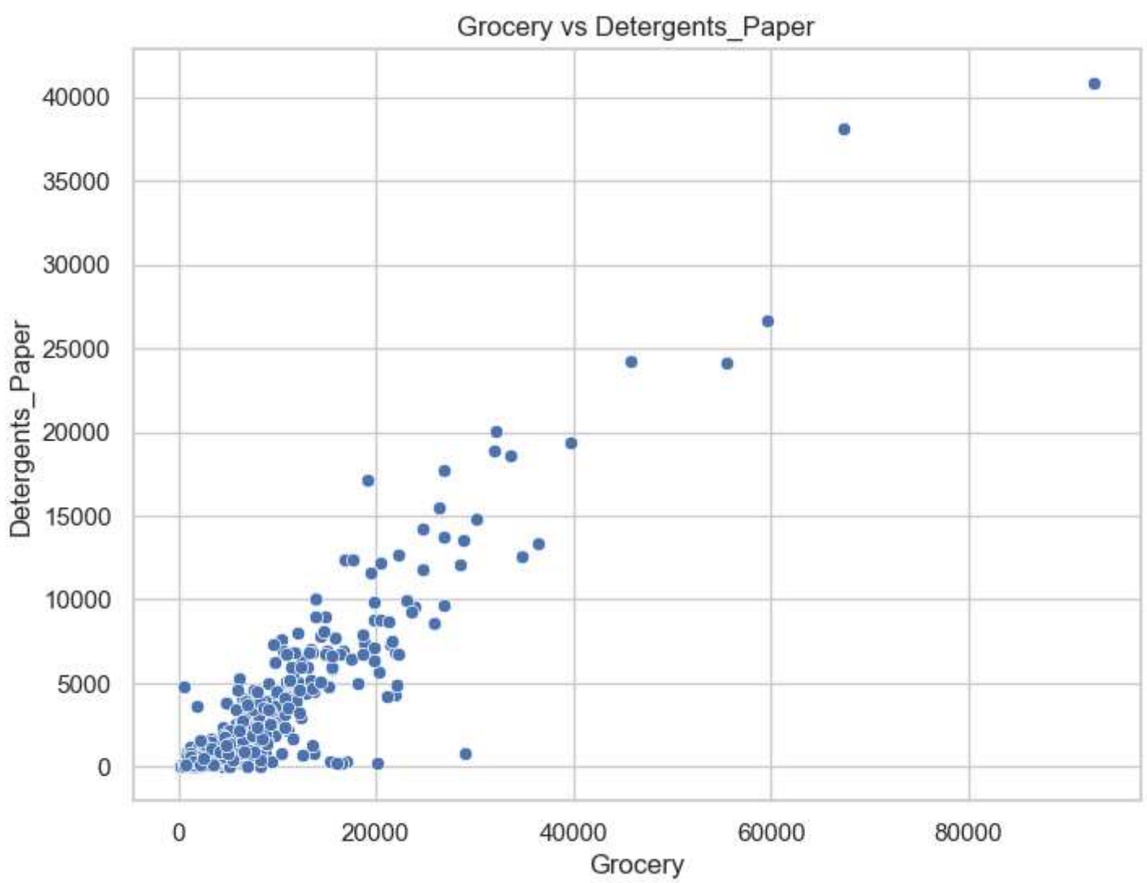
```
Mean spending by Region
```

Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
1	11101.7	5486.4	7403.1	3000.3	2651.1	1354.9
2	9887.7	5088.2	9218.6	4045.4	3687.5	1159.7
3	12533.5	5977.1	7896.4	2944.6	2817.8	1620.6

In [23]:
```
#Scatter Plot
sns.scatterplot(x='Milk',y='Grocery',data=df)
plt.title('Milk vs Grocery')
plt.show()
```



In [24]:
```
sns.scatterplot(x='Grocery',y='Detergents_Paper',data=df)
plt.title('Grocery vs Detergents_Paper')
plt.show()
```



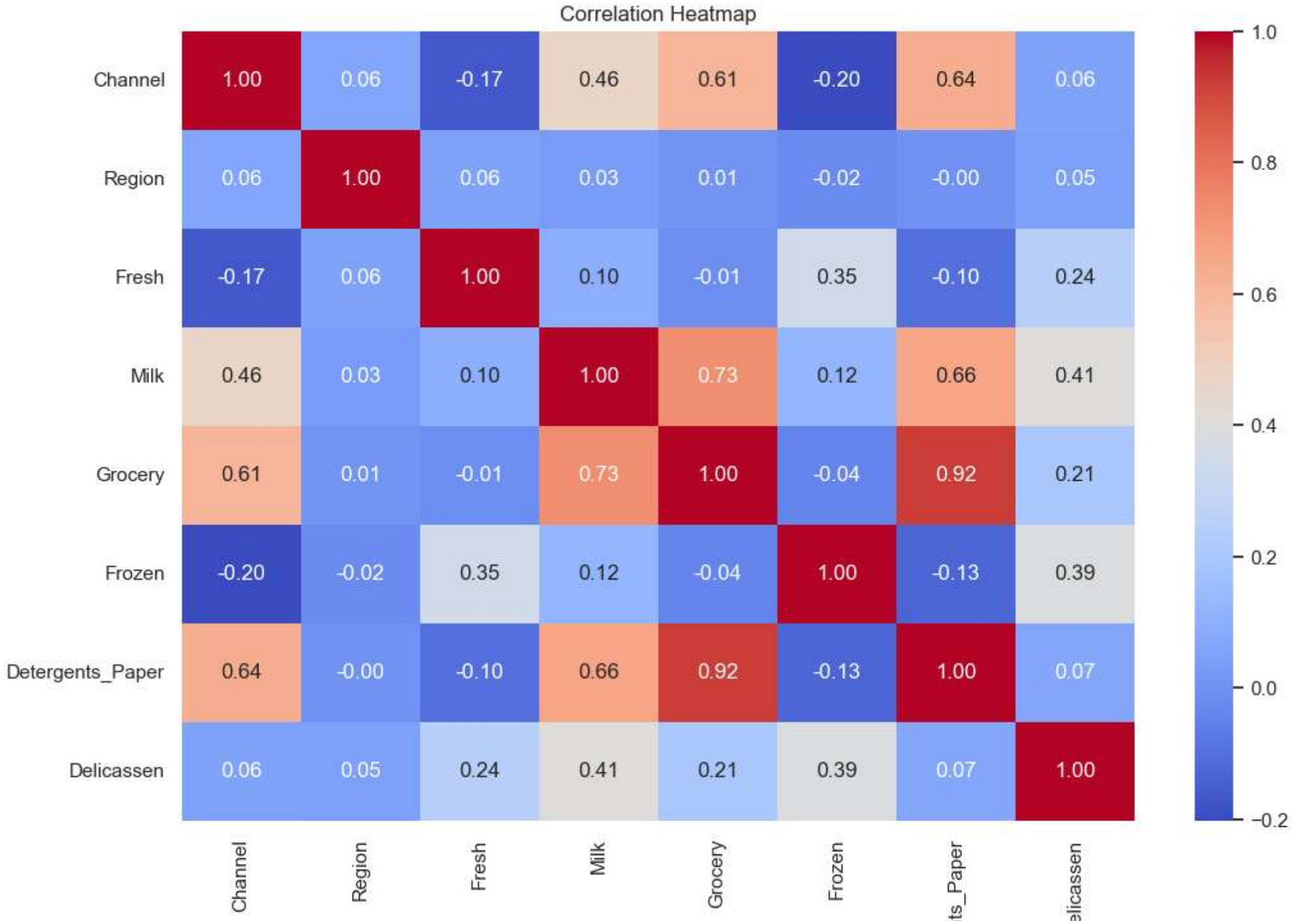
3.Multivariate Analysis

In [25]:
```
plt.figure(figsize=(12,8))

corr=df.corr()

sns.heatmap(corr,annot=True,cmap='coolwarm',fmt='.2f')
```

```
plt.title('Correlation Heatmap')
plt.show()
```

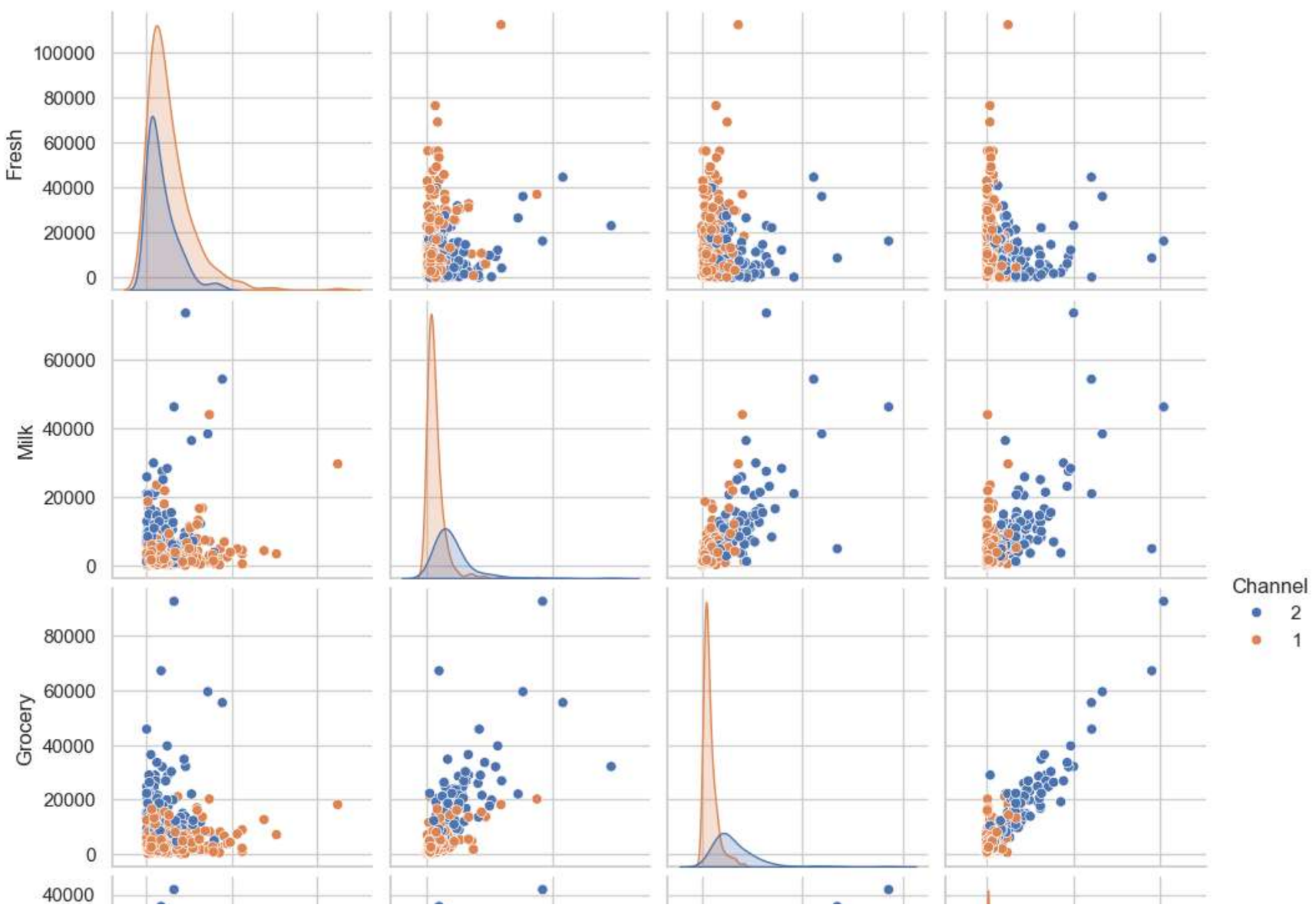

Correlation Heatmap
Channel 1.00 0.06 -0.17 0.46 0.61 -0.20 0.64 0.06
Region 0.06 1.00 0.06 0.03 0.01 -0.02 -0.00 0.05
Fresh -0.17 0.06 1.00 0.10 -0.01 0.35 -0.10 0.24
Milk 0.46 0.03 0.10 1.00 0.73 0.12 0.66 0.41
Grocery 0.61 0.01 -0.01 0.73 1.00 -0.04 0.92 0.21
Frozen -0.20 -0.02 0.35 0.12 -0.04 1.00 -0.13 0.39
Detergents_Paper 0.64 -0.00 -0.10 0.66 0.92 -0.13 1.00 0.07
Delicassen 0.06 0.05 0.24 0.41 0.21 0.39 0.07 1.00
Channel
Region
Fresh
Milk
Grocery
Frozen
ts_Paper
elicassen
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

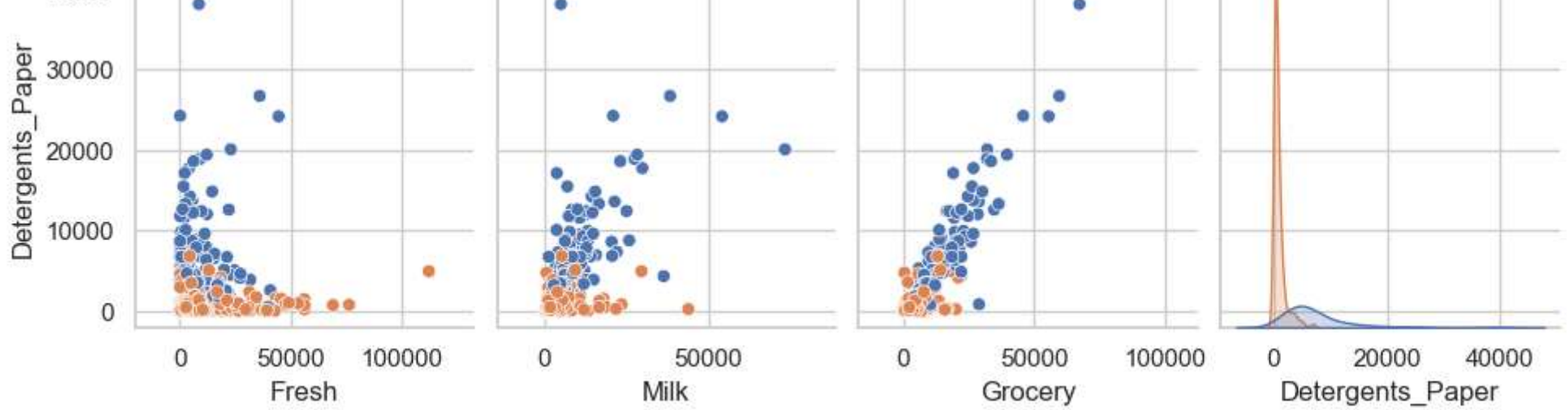


In [26]:
```
# Pair Plot
selected=['Fresh','Milk','Grocery','Detergents_Paper','Channel']

sns.pairplot(df[selected].astype({'Channel':str}),hue='Channel')

plt.show()
```


Fresh
Milk
Grocery
100000
80000
60000
40000
20000
0
Channel
2
1

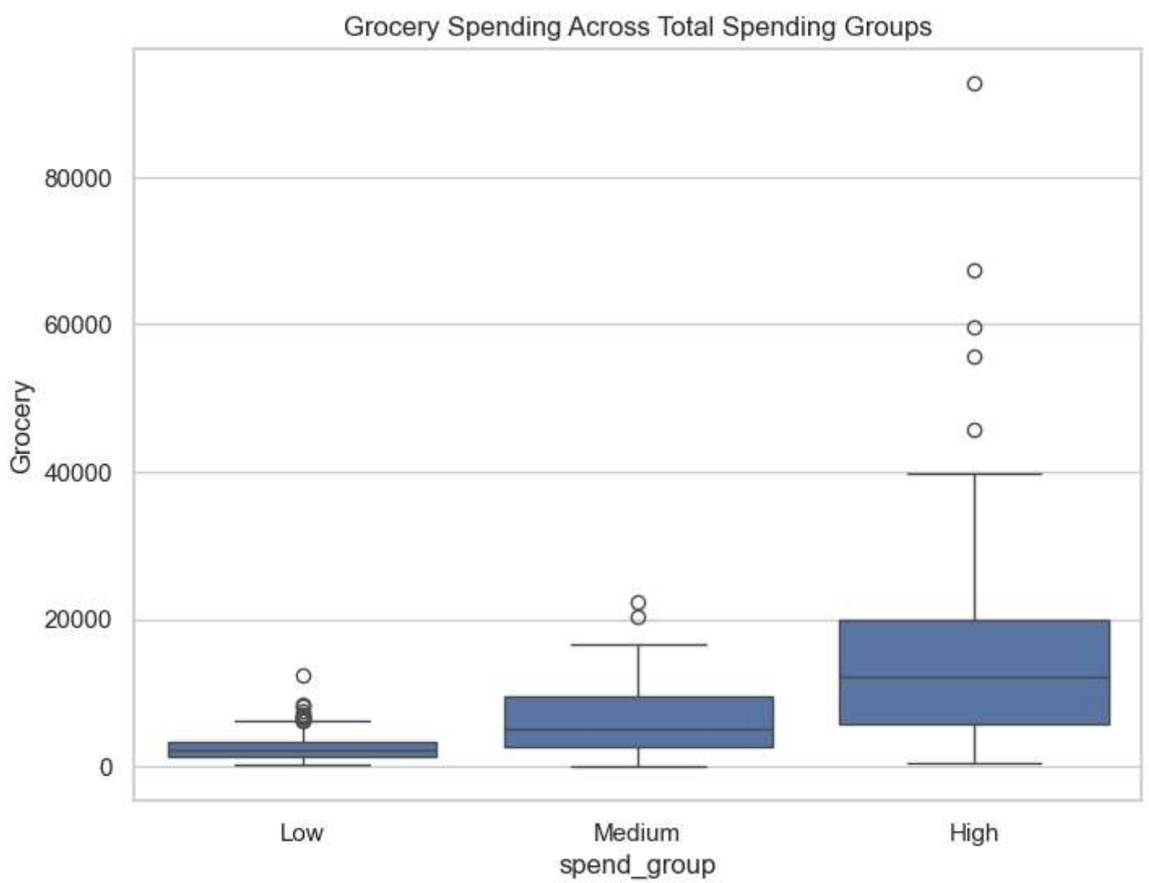


In [27]:
```
#Multivariate Analysis Using Spending Groups
eda_df=df.copy()
eda_df['Total_Spend']=eda_df[spend_features].sum(axis=1)

eda_df['spend_group']=pd.qcut(
    eda_df['Total_Spend'],
    q=3,
    labels=['Low','Medium','High']
)
```

In [28]:
```
sns.boxplot(
    x='spend_group',
    y='Grocery',
    data=eda_df
)

plt.title('Grocery Spending Across Total Spending Groups')
plt.show()
```



Data Preprocessing and Feature Scaling

In [29]:
```
X = df.drop(columns=['Channel', 'Region'])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [30]:
```
import pandas as pd

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=X.columns,
    index=X.index
)

X_scaled_df.head()
```

Out[30]:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	0.052933	0.523568	-0.041115	-0.589367	-0.043569	-0.066339
1	-0.391302	0.544458	0.170318	-0.270136	0.086407	0.089151
2	-0.447029	0.408538	-0.028157	-0.137536	0.133232	2.243293
3	0.100111	-0.624020	-0.392977	0.687144	-0.498588	0.093411
4	0.840239	-0.052396	-0.079356	0.173859	-0.231918	1.299347

In [31]:
```
display( X_scaled_df.head())
```

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	0.052933	0.523568	-0.041115	-0.589367	-0.043569	-0.066339
1	-0.391302	0.544458	0.170318	-0.270136	0.086407	0.089151
2	-0.447029	0.408538	-0.028157	-0.137536	0.133232	2.243293
3	0.100111	-0.624020	-0.392977	0.687144	-0.498588	0.093411
4	0.840239	-0.052396	-0.079356	0.173859	-0.231918	1.299347

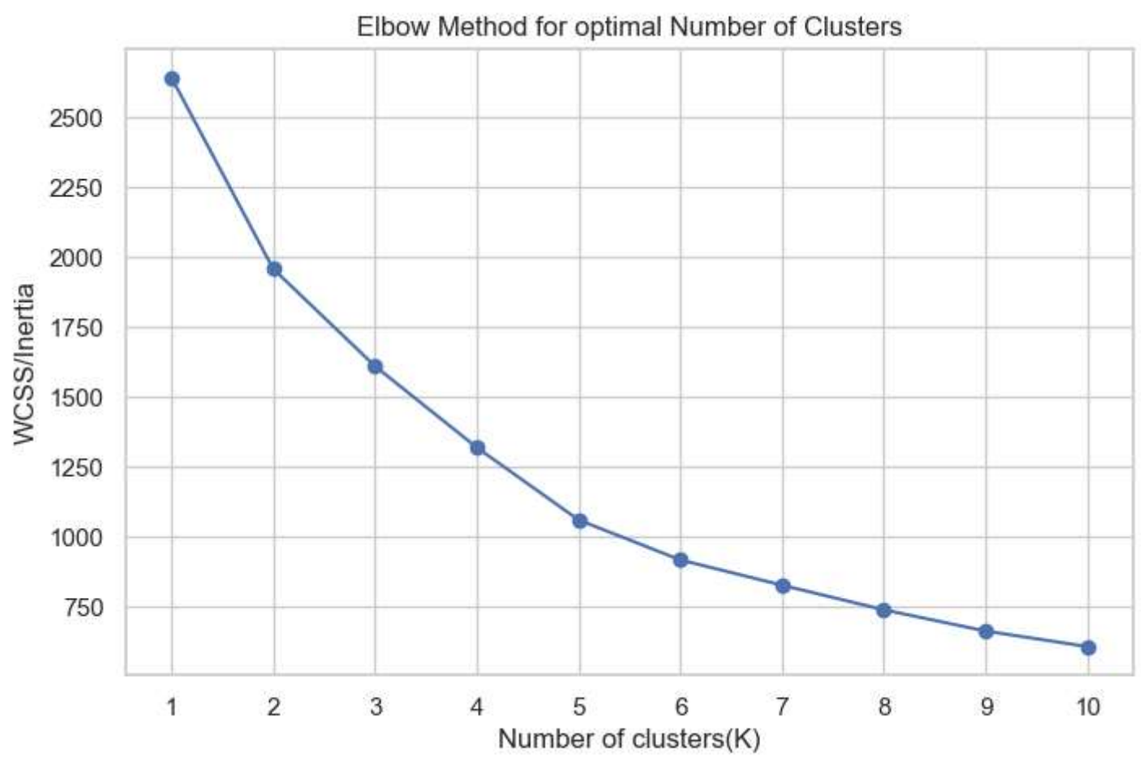
In [32]:
```
print(X_scaled_df.mean().round())
```

```
Fresh              -0.0
Milk                0.0
Grocery            -0.0
Frozen              0.0
Detergents_Paper    0.0
Delicassen         -0.0
dtype: float64
```

In [33]:
```
print("\n Scaled feature sd ")
print(X_scaled_df.std(ddof=0).round(3))
```

```
 Scaled feature sd
Fresh              1.0
Milk               1.0
Grocery            1.0
Frozen             1.0
Detergents_Paper   1.0
Delicassen         1.0
dtype: float64
```

In [34]:
```
wccs=[]
k_values=range(1,11)
for k in k_values:
    kmeans=KMeans(n_clusters=k,random_state=42,n_init=10)
    kmeans.fit(X_scaled_df)
    wccs.append(kmeans.inertia_)
plt.figure(figsize=(8,5))
plt.plot(list(k_values),wccs,marker="o")
plt.xlabel("Number of clusters(K)")
plt.ylabel("WCSS/Inertia")
plt.title("Elbow Method for optimal Number of Clusters")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()
```



In [35]:
```
silhouette_values = []
for k in range(2, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled_df)
    score = silhouette_score(X_scaled_df, labels)
    silhouette_values.append(score)
silhouette_table = pd.DataFrame({
    "Number of Clusters": list(range(2, 11)),
    "Silhouette Score": silhouette_values
})
display(silhouette_table.round(4))
```

	Number of Clusters	Silhouette Score
0	2	0.5472
1	3	0.5483
2	4	0.3485
3	5	0.3690
4	6	0.3782
5	7	0.3343
6	8	0.3201
7	9	0.3090
8	10	0.3112

In [36]:
```
suggested_k=int(
    silhouette_table.loc[silhouette_table["Silhouette Score"].idxmax(),"Number of Clusters"]
)
print("K suggested by the highest silhouette score:",suggested_k)
```

```
K suggested by the highest silhouette score: 3
```

In [37]:
```
K=suggested_k
kmeans_model=KMeans(n_clusters=K,random_state=42,n_init=10)
kmean_labels=kmeans_model.fit(X_scaled_df)
```

In [38]:
```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmean_labels = kmeans.fit_predict(X_scaled_df)
df_kmeans = df.copy()
df_kmeans["KMeans_Cluster"] = kmean_labels
print("K-Means clustering complete!")
print("\nNumber of customers in each cluster:")
print(df_kmeans["KMeans_Cluster"].value_counts().sort_index())
print("\nSample records with cluster labels:")
display(df_kmeans.head())
```

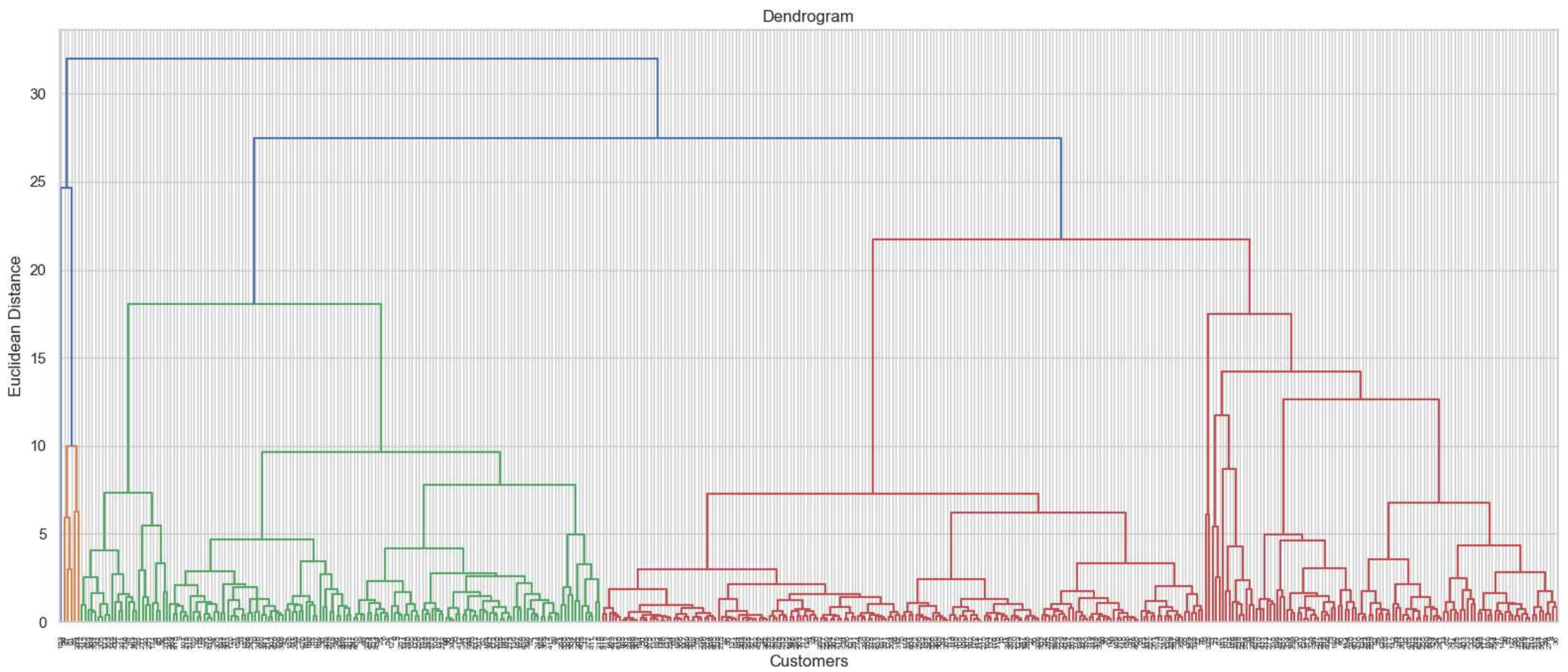
```
K-Means clustering complete!

Number of customers in each cluster:
KMeans_Cluster
0     45
1    393
2      2
Name: count, dtype: int64

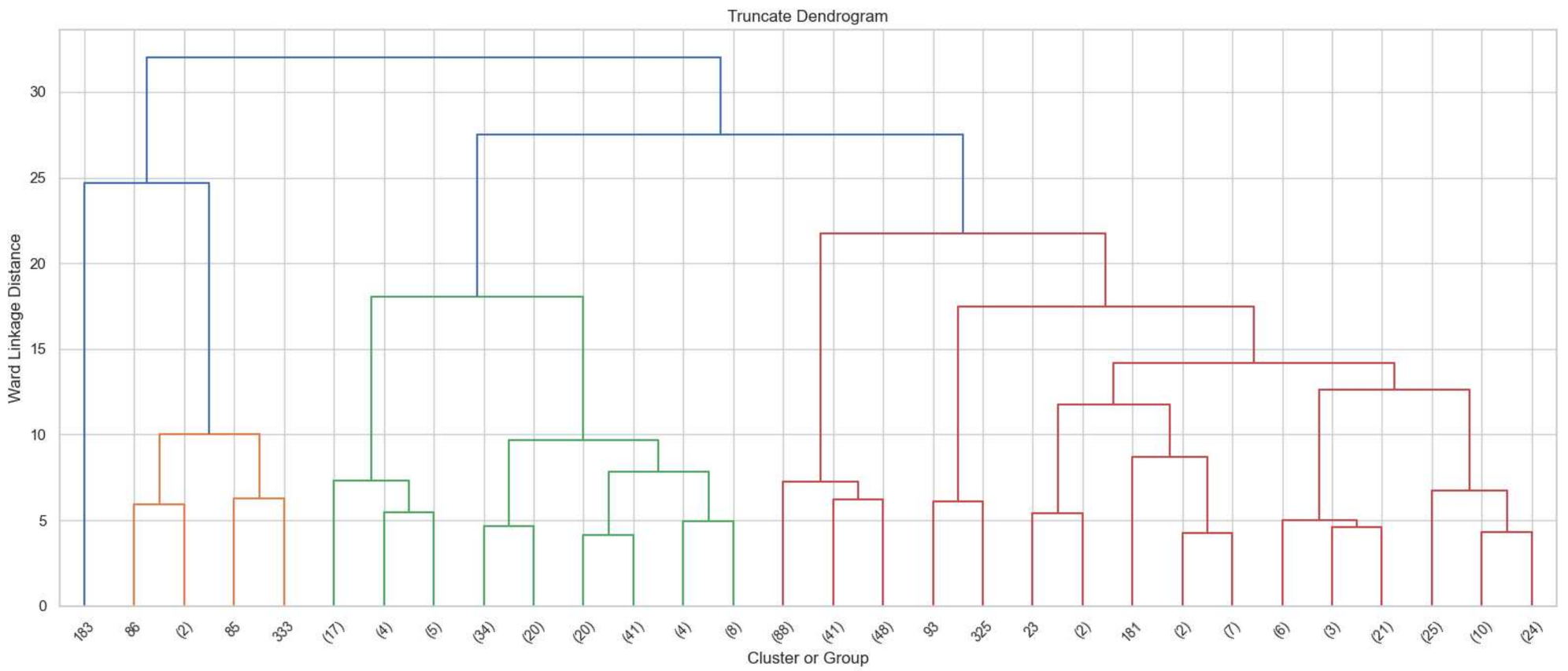
Sample records with cluster labels:
```

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	KMeans_Cluster
0	2	3	12669	9656	7561	214	2674	1338	1
1	2	3	7057	9810	9568	1762	3293	1776	1
2	2	3	6353	8808	7684	2405	3516	7844	1
3	1	3	13265	1196	4221	6404	507	1788	1
4	2	3	22615	5410	7198	3915	1777	5185	1

In [39]:
```
Z=linkage(X_scaled_df,method="ward",metric="euclidean")
plt.figure(figsize=(16,7))
dendrogram(Z,leaf_rotation=90,leaf_font_size=6)
plt.title("Dendrogram")
plt.xlabel("Customers")
plt.ylabel("Euclidean Distance")
plt.tight_layout()
plt.show()
```



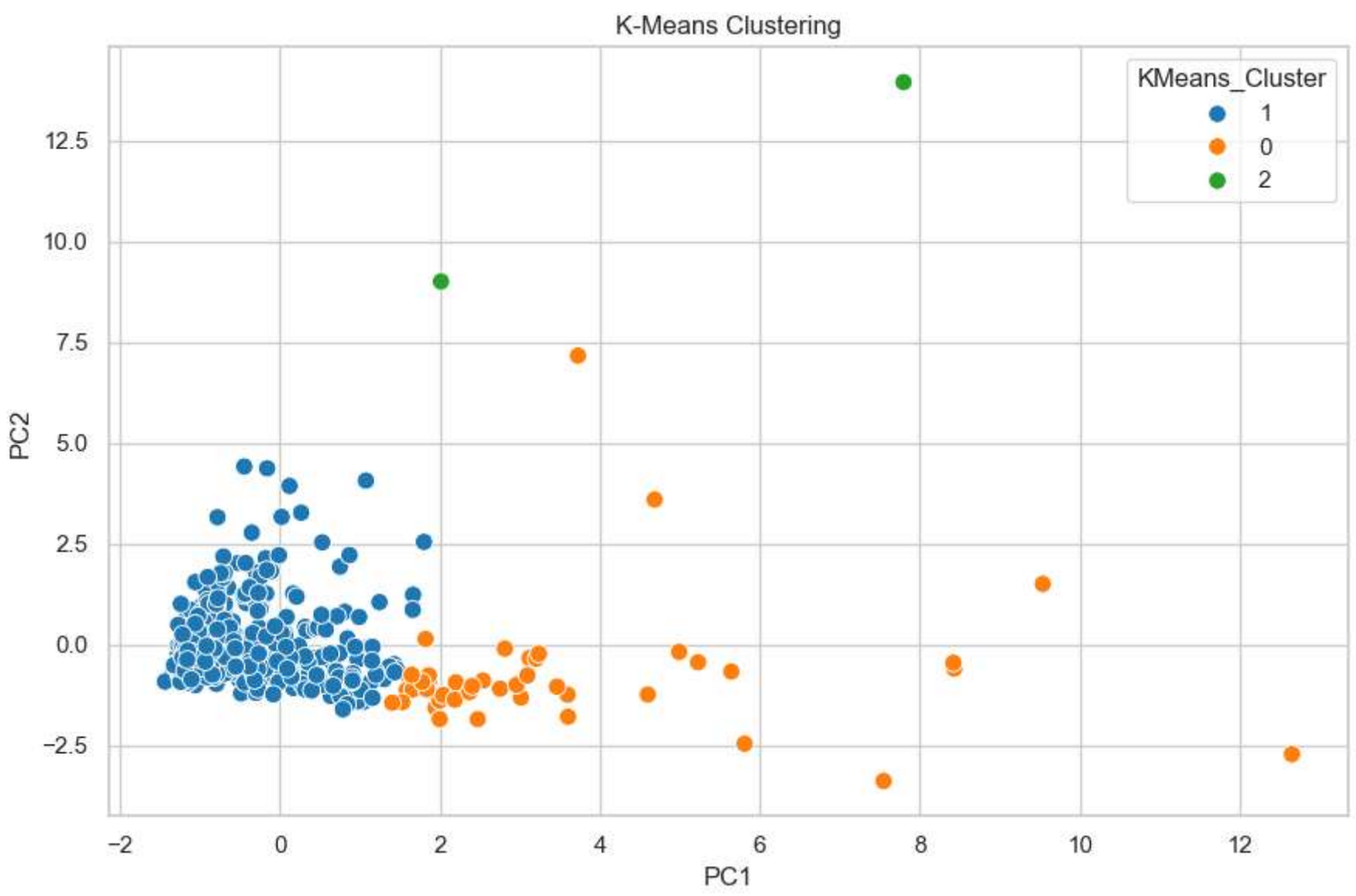
In [40]:
```
plt.figure(figsize=(16,7))
dendrogram(Z,truncate_mode="lastp",p=30,show_leaf_counts=True,leaf_rotation=45)
plt.title("Truncate Dendrogram")
plt.xlabel("Cluster or Group")
plt.ylabel("Ward Linkage Distance")
plt.tight_layout()
plt.show()
```



In [41]:
```
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled_df)
print(
    "Variance exlained by two PCA coponent:",
    round(pca.explained_variance_ratio_.sum()*100,2),
    "%"
)
plot_df=pd.DataFrame({"PC1": X_pca[:,0],"PC2": X_pca[:,1],
                      "KMeans_Cluster":kmean_labels.astype(str)})
```

```
Variance exlained by two PCA coponent: 72.46 %
```

In [42]:
```
plt.figure(figsize=(9,6))
sns.scatterplot(data=plot_df,x="PC1",y="PC2",hue="KMeans_Cluster",palette="tab10",s=70)
plt.title("K-Means Clustering")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.tight_layout()
plt.show()
```



In [43]:
```python
def evaluate_clustering(X_data,labels,model_name):
  return{
      "Algorithm":model_name,
      "Number Of Clusters":len(np.unique(labels)),
      "Silhouette Score":silhouette_score(X_data,labels)
  }
```

In [44]:
```
hierarchical_model = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
hierarchical_labels = hierarchical_model.fit_predict(X_scaled_df)

comparision = pd.DataFrame([
    evaluate_clustering(X_scaled_df, kmean_labels, "K-Means"),
    evaluate_clustering(X_scaled_df, hierarchical_labels, "Hierarchical Clustering")
])

display(comparision.round(4))
```

	Algorithm	Number Of Clusters	Silhouette Score
0	K-Means	3	0.5483
1	Hierarchical Clustering	3	0.2646

In [45]:
```
cluster_sizes=pd.DataFrame({
    "K-Means":pd.Series(kmean_labels).value_counts().sort_index(),
    "Hierarchical":pd.Series(hierarchical_labels).value_counts().sort_index()
})
display(cluster_sizes)
```

	K-Means	Hierarchical
0	45	6
1	393	281
2	2	153

In [46]:
```
features = list(X_scaled_df.columns)
kmeans_profile=df[features].copy()
kmeans_profile["Cluster"]=kmean_labels
kmeans_means=kmeans_profile.groupby("Cluster").mean()
display(kmeans_means)
```

Cluster	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
0	10440.933333	19386.422222	28656.088889	2190.244444	13327.800000	2374.200000
1	12062.913486	4115.099237	5534.966921	2940.676845	1696.170483	1299.114504
2	34782.000000	30367.000000	16898.000000	48701.500000	755.500000	26776.000000

In [ ]: